

Guide Technique — ViteComm

ViteComm — Guide Fonctionnel et Technique

Plateforme de commerce local au Bénin | MVP Académique | Juin 2026

Stack : React 19 + Vite 8 + Tailwind CSS 4 + Node.js + Express + Prisma + MySQL/MariaDB

1. Architecture Globale

ViteComm adopte une **architecture client-serveur séparée** (frontend déconnecté du backend) avec une API RESTful. Le frontend est une application SPA (Single Page Application) React déployée sur Vercel. Le backend est un serveur Node.js/Express hébergé sur AlwaysData, exposant une API REST consumed par le frontend via `fetch`.

Approche architecturale : Architecture modulaire en couches (layers) côté backend (`routes` → `controllers` → `services` → `prisma`) et composants React côté frontend. Pas de state management global (Redux/Zustand) — uniquement React Context pour l'auth et le thème.

Couche	Backend (Node.js)	Frontend (React)
Présentation	Routes Express (<code>/api/*</code>)	Pages React + composants
Logique métier	Controllers + Services	Context (Auth, Theme) + hooks
Accès données	Prisma ORM → MySQL/MariaDB	Service API (<code>fetch</code>)
Sécurité	JWT + RBAC middleware + HMAC webhook	Token localStorage + role guard
Déploiement	AlwaysData (Node.js host)	Vercel (SPA hosting)

2. Structure des Dossiers

2.1 Backend (`backend/`)

```
backend/ ├── src/ | ├── config/ | | └─ db.js | ├── controllers/ | | ├── adminController.js | | ├── authController.js | | ├── clientController.js | | ├── livreurController.js | | ├── paymentController.js | | └─ vendeurController.js | ├── middleware/ | | ├── auth.js | | └─ upload.js | ├── routes/ | | ├── adminRoutes.js | | ├── authRoutes.js | | ├── clientRoutes.js | | ├── livreurRoutes.js | | ├── paymentRoutes.js | | └─ vendeurRoutes.js | ├── services/ | | ├── fedapayService.js | | └─ mail.js | ├── utils/ | | ├── errors.js | | └─ vendorQR.js | └─ index.js | ├── prisma/ | ├── schema.prisma | ├── prisma.config.ts | └─ seed.js | ├── uploads/ | ├── package.json | ├── package-lock.json | └─ .env
```

2.2 Frontend (`frontend/`)

```

frontend/ ├── src/ | ├── App.jsx | ├── main.jsx | ├── index.css | ├── context/ | | ├── AuthContext.jsx | | | |
ThemeContext.jsx | | services/ | | | └── api.js | | components/ | | | ├── ErrorBoundary.jsx | | | ├── Footer.jsx | | | |
InstallPrompt.jsx | | | MobileDrawer.jsx | | | ├── GoogleSignInButton.jsx | | | ├── GoogleRoleSelection.jsx | | | | client/ |
| | | ClientLayout.jsx | | | | └── BottomNav.jsx | | | vendeur/ | | | | ├── VendeurLayout.jsx | | | | | └── BottomNav.jsx | |
└── livreur/ | | | ├── LivreurLayout.jsx | | | | └── BottomNav.jsx | | | pages/ | | | ├── Accueil/ | | | | └── Accueil.jsx | | | auth/ |
| | | ├── Connexion.jsx | | | ├── Inscription.jsx | | | ├── ForgotPassword.jsx | | | ├── CGU.jsx | | | | PolitiqueConfidentialite.jsx
| | | client/ | | | | ├── AccueilClient.jsx | | | | ├── Catalogue.jsx | | | | ├── Panier.jsx | | | | | SelectionLivreur.jsx | | | |
SuiviCommande.jsx | | | Profil.jsx | | | ├── MesCommandes.jsx | | | ├── MesFactures.jsx | | | | Evaluation.jsx | | | |
Inspection.jsx | | | PaiementClient.jsx | | | ├── MarcheDetail.jsx | | | | UserDetail.jsx | | | vendeur/ | | | |
Dashboard.jsx | | | Catalogues.jsx | | | ├── Commandes.jsx | | | | Retour.jsx | | | | Statistiques.jsx | | | | Factures.jsx
| | | Signalement.jsx | | | | VendeurProfil.jsx | | | livreur/ | | | | Livreur.jsx | | | | Commandes.jsx | | | |
Gains.jsx | | | Historique.jsx | | | ├── Retours.jsx | | | | LivreurProfil.jsx | | | admin/ | | | | Admin.jsx | | | public/ |
| vitecomm.png | | icon-192x192.png | | icon-512x512.png | | icon-maskable-512x512.png | | favicon.svg | | |
icons.svg | | dist/ | | vercel.json | | vite.config.js | | package.json | | package-lock.json | | .env

```

3. Variables d'Environnement

3.1 Backend (backend/.env)

Variable	Description	Exemple
APP_ENV	Environnement d'exécution	local development production
APP_URL	URL publique du backend	https://vitecomm.alwaysdata.net
FRONTEND_URL	URL du frontend (redirects Google OAuth, CORS)	https://vitecomm.vercel.app
APP_DEBUG	Afficher les erreurs détaillées	true (dev) false (prod)
CORS_ORIGINS	Origines autorisées (virgule)	https://vitecomm.vercel.app
DATABASE_URL	Chaîne de connexion MySQL	mysql://user:pass@host:3306/db
PORT	Port d'écoute (alwaysdata injecte 8100)	5000 (local) 8100 (prod)
JWT_SECRET	Clé secrète pour signer les JWT	Chaîne aléatoire longue
JWT_EXPIRES_IN	Durée de vie du token	7d
MAIL_HOST	Serveur SMTP	smtp.gmail.com
MAIL_PORT	Port SMTP	587
MAIL_USERNAME	Email expéditeur	viteecomm@gmail.com
MAIL_PASSWORD	Mot de passe application Gmail	yquwhfcebmnawomb
GOOGLE_CLIENT_ID	ID client Google OAuth	apps.googleusercontent.com

GOOGLE_CLIENT_SECRET	Secret client Google	GOCSPX-...
GOOGLE_REDIRECT_URL	URL callback Google OAuth	https://vitecomm.alwaysdata.net/api/auth/google/callback
FEDAPAY_SECRET_KEY	Clé API FedaPay	sk_sandbox...
FEDAPAY_WEBHOOK_SECRET	Secret webhook FedaPay (HMAC)	wh_sandbox...
FEDAPAY_ENVIRONMENT	Environnement FedaPay	sandbox production
VENDOR_QR_SECRET	Clé HMAC pour signer les QR codes vendeur	Chaîne secrète

3.2 Frontend (frontend/.env)

Variable	Description	Exemple
VITE_API_BASE_URL	Base URL de l'API backend	/api (dev/proxy) https://vitecomm.alwaysdata.net/api (prod)
VITE_BACKEND_HOST	Hôte backend (proxy dev uniquement)	127.0.0.1
VITE_BACKEND_PORT	Port backend (proxy dev uniquement)	5000
VITE_GOOGLE_CLIENT_ID	ID client Google (même que backend)	...apps.googleusercontent.com

Note : `VITE_BACKEND_HOST` et `VITE_BACKEND_PORT` ne sont utilisés que par le proxy Vite en développement. En production (Vercel), le frontend appelle le backend directement via `VITE_API_BASE_URL` .

4. Lancement en Local

4.1 Prérequis

- Node.js ≥ 18
- MySQL/MariaDB local (port 3306)
- Git

4.2 Backend

```
# 1. Cloner le dépôt
git clone <repo-url> && cd ViteComm

# 2. Installer les dépendances
cd backend && npm install

# 3. Créer la base de données
mysql -u root -e "CREATE DATABASE vitecomm;"

# 4. Configurer .env (copier .env.example et remplir)

# 5. Générer le client Prisma + appliquer les migrations
npx prisma generate
npx prisma db push

# 6. (Optionnel) Peupler la base
node prisma/seed.js

# 7. Lancer le serveur
npm run dev      # → nodemon, port 5000
```

4.3 Frontend

```
# 1. Depuis la racine du projet
cd frontend

# 2. Installer les dépendances
npm install

# 3. Configurer .env (VITE_API_BASE_URL=/api, VITE_BACKEND_HOST=127.0.0.1)

# 4. Lancer le serveur de dev
npm run dev    # → Vite, port 5173
```

Proxy automatique : En dev, les requêtes `/api/*` et `/uploads/*` sont automatiquement proxyées vers le backend (`127.0.0.1:5000`) grâce à la config dans `vite.config.js` . Pas besoin de CORS en dev.

5. Schema Prisma — Base de Données

Le schéma Prisma définit **22 modèles** mappés sur MySQL/MariaDB. La base suit une architecture de **spécialisation** : une table parente `Utilisateur` avec des tables enfants `Client` , `Vendeur` , `Livreur` .

5.1 Modèles Principaux

Modèle	Rôle	Champs Clés
Utilisateur	Table parente – tous les utilisateurs	id_user (PK), nom , prenom , email (unique), mot_de_passe (hash bcrypt), statut_compte , est_admin , auth_provider ("local" "google")
Client	Spécialisation client	id_user (PK/FK cascade), adresse_livraison
Vendeur	Spécialisation vendeur	id_user (PK/FK), nom_etablissement , id_marche (FK), score_reputation (défaut 0)
Livreur	Spécialisation livreur	id_user (PK/FK), type_vehicule , immatriculation , score_reputation
Marche	Marché physique	id_marche , nom (unique), latitude , longitude
Produit	Article en vente	id_produit , nom , prix_reference , stock_disponible , unite (kg/tas/pièce), id_user_vendeur (FK), id_categorie (FK)
Panier	Panier client (1 par client)	id_panier , id_user_client (unique FK)
DetailPanier	Ligne de panier	Composite PK (id_panier, id_produit) , quantite
Commande	Commande client	id_commande , statut (défaut "En attente"), code_verification (6 car. aléatoire), total_marchandises , frais_livraison , commission
DetailCommande	Ligne de commande (entité ternaire)	Composite PK (id_commande, id_produit) , quantite_commandee , prix_vente_applique (prix figé), statut_acceptation
Livraison	Affectation livreur	id_livraison , statut_livraison , id_commande (unique FK), id_user_livreur (FK)
PreuveCollecte	Preuve de collecte photo	id_preuve , date_heure , statut_validation , id_commande (FK)

MediaPreuve	Photo/vidéo jointe à une preuve	id_media , url_media , type_media ("photo" "vidéo"), id_preuve (FK)
Litige	Contestation client	id_litige , statut ("Ouvert"/"Accepté"/"Rejeté"), decision_admin , montant_rembourse , id_livraison (FK)
Feedback	Évaluation	id_feedback , note (1-5), type_feedback ("LIVREUR" "VENDEUR")
Signalement	Signalement universel	id_signalement , motif , id_auteur (FK), id_cible (FK)
Facture	Facture (source de vérité financière)	id_facture , montant_total_du , statut_paiement , id_commande (FK)
PaiementTransaction	Transaction en ligne (Fedapay)	transaction_id (unique), montant , statut ("pending"/"approved"/"declined")
VerificationToken	Code vérification inscription	email , code (6 chiffres), data (JSON du formulaire), expires_at
PasswordResetToken	Code réinitialisation MDP	email , code , expires_at
HistoriquePrix	Audit des changements de prix	prix , date_modification , id_produit (FK)
DisponibiliteLivreur	Historique disponibilité livreur	est_disponible , distance_marche , heure_debut_dispo , heure_fin_dispo , id_user_livreur (FK)
BonDeLivraison	Bon de livraison (logistique)	statut_bon ("EN_ATTENTE"/"SIGNE"), id_livraison (FK)

5.2 Relations Clés

- **Spécialisation** : `Utilisateur` 1:0..1 `Client` | `Vendeur` | `Livreur` (cascade delete)
- **Panier** : `Client` 1:1 `Panier` 1:N `DetailPanier`
- **Commande** : `Commande` N:1 `Client` , `DetailCommande` est l'entité ternaire entre `Commande` et `Produit`
- **Livraison** : `Livraison` 1:1 `Commande` (unique), N:1 `Livreur`
- **Preuves** : `PreuveCollecte` 1:N `MediaPreuve`
- **Signalement** : `Signalement` N:1 `Utilisateur` (auteur) + N:1 `Utilisateur` (cible)

5.3 Flux de Commande (State Machine)

En attente → Validée par vendeur → Assignée au livreur → En cours de collecte
→ Collectée (QR scanné) → En cours de livraison → Inspection client
→ Livrée (si acceptée) / Litige (si rejetée) → Facturée → Payée → Feedback

6. Authentification JWT

6.1 Comment ça marche

ViteComm utilise **JSON Web Tokens (JWT)** pour l'authentification stateless. Pas de sessions côté serveur.

- 1 **Inscription** : L'utilisateur remplit le formulaire → le backend envoie un email avec un **code à 6 chiffres** → l'utilisateur vérifie son email → le compte est créé et un JWT est émis.

- 2 **Connexion** : Email/mdp → le backend vérifie le hash bcrypt → émet un JWT signé avec `JWT_SECRET` → le frontend stocke le token dans `localStorage` (clé `vc_token`).
- 3 **Requêtes authentifiées** : Le frontend envoie `Authorization: Bearer <token>` dans chaque requête → le middleware `requireAuth` vérifie la signature, charge l'utilisateur en BDD, déduit le rôle, et attache `req.user`.

6.2 Payload du JWT

```
{
  "id_user": 42,
  "role": "client" // déduit dynamiquement, pas stocké en BDD
}
```

6.3 Expiration

Par défaut : `7 jours` (configurable via `JWT_EXPIRES_IN`). Le frontend ne gère pas le refresh automatique — un 401 redirige vers `/connect`.

6.4 Dérivation du Rôle

Le rôle **n'est pas stocké** dans la table `Utilisateur`. Il est déduit dynamiquement par le middleware :

```
if (user.client) → rôle = "client"
if (user.vendeur) → rôle = "vendeur"
if (user.livreur) → rôle = "livreur"
sinon → rôle = "admin"
```

7. Contrôle d'Accès Basé sur les Rôles (RBAC)

7.1 Les 4 Rôles

Rôle	Description	Accès
client	Acheteur	Catalogue, panier, commandes, inspection, paiement, feedback
vendeur	Commerçant	Dashboard, produits CRUD, commandes, retours, factures, statistiques
livreur	Transporteur	Dashboard, disponibilité, livraisons, gains, historique
admin	Administrateur	Tout : users, produits, marchés, litiges, signalements, analytics

7.2 Implémentation

Deux middlewares composent le RBAC :

```

// middleware/auth.js

// 1. requireAuth – vérifie le JWT et charge l'utilisateur
function requireAuth(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  const decoded = jwt.verify(token, JWT_SECRET);
  const user = await prisma.utilisateur.findUnique({
    where: { id_user: decoded.id_user },
    include: { client: true, vendeur: true, livreur: true }
  });
  // Déduit le rôle dynamiquement
  req.user = { id_user, email, nom, prenom, role };
  next();
}

// 2. requireRole – vérifie que le rôle est autorisé
function requireRole(allowedRoles) {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: "Accès refusé" });
    }
    next();
  };
}

```

7.3 Application par Route

Préfixe	Méthode	Rôles Autorisés
<code>/api/auth/*</code>	Mélangé	Public (inscription, connexion) + <code>requireAuth</code> (profil)
<code>/api/client/*</code>	Toutes	<code>requireAuth</code> + <code>requireRole(['client'])</code>
<code>/api/vendor/*</code>	Toutes	<code>requireAuth</code> + <code>requireRole(['vendeur'])</code>
<code>/api/livreur/*</code>	Toutes	<code>requireAuth</code> + <code>requireRole(['livreur'])</code>
<code>/api/admin/*</code>	Toutes	<code>requireAuth</code> + <code>requireRole(['admin'])</code>
<code>/api/webhooks/*</code>	POST	Public (vérifié par HMAC signature)

Sécurité : Un vendeur ne peut jamais accéder aux routes client, et inversement. Le RBAC est vérifié **côté serveur** à chaque requête. Le frontend masque simplement l'UI, mais la sécurité finale est toujours backend.

8. Authentification Google OAuth

8.1 Deux flux implémentés

Flux 1 : Access Token (frontend → backend)

- 1 Le frontend utilise `useGoogleLogin` (flux implicite) pour obtenir un `access_token` Google.
- 2 Le frontend envoie le token au backend : `POST /api/auth/google` avec `{ credential: access_token }`.

- 3 Le backend appelle `https://www.googleapis.com/oauth2/v3/userinfo` avec le token pour récupérer `email` , `given_name` , `family_name` .
- 4 **Si nouvel utilisateur** : création automatique en tant que `client` avec `auth_provider: 'google'` , mot de passe aléatoire hashé, panier créé. Le flag `is_new_google_user: true` est renvoyé.
- 5 **Si utilisateur existant avec `auth_provider: 'local'`** : erreur 403 — "Utilisez la connexion email/mot de passe".
- 6 Le backend émet un JWT et renvoie les infos utilisateur + token.

Flux 2 : Authorization Code (redirection serveur)

- 1 `GET /api/auth/google` redirige vers l'écran de consentement Google.
- 2 Google redirige vers `/api/auth/google/callback?code=...` .
- 3 Le backend échange le code contre des tokens, vérifie l'ID token, crée/l'utilisateur.
- 4 Redirige vers `FRONTEND_URL/auth/google/callback?token=...&user=...&new=...` .

8.2 Complétion du profil après Google

Si `is_new_google_user` est vrai, le frontend affiche le composant `GoogleRoleSelection` qui permet de choisir un rôle (client/vendeur/livreur) et de remplir les informations spécifiques (nom d'établissement, type de véhicule, etc.). L'appel `POST /api/auth/google/complete-registration` crée la spécialisation appropriée.

9. Système de Mail (SMTP)

9.1 Configuration

- **Librairie** : `nodemailer`
- **Serveur** : Gmail SMTP (`smtp.gmail.com:587` , TLS)
- **Auth** : `MAIL_USERNAME` + `MAIL_PASSWORD` (mot de passe application Gmail)
- **Expéditeur** : "ViteComm" <`viteecomm@gmail.com`>

9.2 Emails envoyés

Type	Déclencheur	Contenu
Code de vérification	Inscription (<code>POST /api/auth/register</code>)	Code à 6 chiffres dans un email HTML stylé. Expire après 10 minutes.
Réinitialisation MDP	Mot de passe oublié (<code>POST /api/auth/forgot-password</code>)	Code à 6 chiffres. Expire après 10 minutes.

9.3 Flux d'inscription complet

- 1 `POST /api/auth/register` — valide les champs, génère un code (6 chiffres) + un token UUID, stocke un `VerificationToken` (data = JSON du formulaire complet), envoie l'email.
- 2 L'utilisateur saisit le code dans l'interface → `POST /api/auth/verify-email` .
- 3 Le backend vérifie le code + token, crée l'utilisateur + la spécialisation dans une transaction Prisma, émet un JWT.
- 4 `POST /api/auth/resent-code` — régénère un code, étend l'expiration, renvoie l'email.

Point important : L'utilisateur n'est **pas créé en BDD** avant la vérification email. Les données du formulaire sont stockées en JSON dans `VerificationToken.data` le temps de la vérification. Cela évite les comptes fantômes.

10. Système de Verification par QR Code

Le système de vérification ViteComm repose sur une **chaîne de confiance à trois niveaux** (Client → Vendeur → Livreur) garantissant que le livreur a bien récupéré les produits chez le vendeur et les a remis au bon client.

10.1 Les trois QR codes

QR Code	Généré par	Scanné par	Sécurité
Code client	Système (6 caractères aléatoires)	Le livreur le saisit manuellement	Code simple, pas signé
QR Vendeur	Vendeur via <code>POST /api/vendor/orders/:id/qrcode</code>	Le livreur scanne	HMAC-SHA256, payload = <code>orderId:clientId:timestamp</code> , expire 24h
QR Finalisation	Client via <code>GET /api/client/orders/:id/finalize-qrcode</code>	Le livreur scanne	HMAC-SHA256, chaîne = <code>orderId:clientId:vendorQRHash:timestamp</code> , expire 1h

10.2 Flux complet de vérification

- Création commande :** Le système génère un `code_verification` (6 car. aléatoires). Le client le voit dans le suivi de commande.
- Le livreur accepte :** Il saisit le code client pour prouver qu'il a bien reçu les informations.

- 3 **Le vendeur génère son QR** : Il signe un token HMAC contenant l'id de la commande + le code client + un timestamp. Le livreur scanne ce QR chez le vendeur.
- 4 **Vérification serveur** : Le backend vérifie la signature HMAC, l'id de la commande, et la correspondance du code client. La preuve photo est enregistrée obligatoirement.
- 5 **Livraison au client** : Le client génère un QR de finalisation qui inclut le hash du QR vendeur (chaîne de confiance). Le livreur scanne ce QR pour finaliser.

10.3 Code source — Sign HMAC

```
// utils/vendorQR.js
import crypto from 'crypto';

const SECRET = process.env.VENDOR_QR_SECRET;

export function generateVendorQRToken(orderId, clientCode) {
  const payload = `${orderId}:${clientCode}:${Date.now()}`;
  const signature = crypto.createHmac('sha256', SECRET).update(payload).digest('hex');
  return JSON.stringify({ v: 1, t: "vendor", p: payload, s: signature });
}

export function verifyVendorQRToken(scannedText) {
  const { p, s } = JSON.parse(scannedText);
  const expected = crypto.createHmac('sha256', SECRET).update(p).digest('hex');
  if (!crypto.timingSafeEqual(Buffer.from(s), Buffer.from(expected))) throw new Error("Signature invalide");
  // Vérifie l'expiration (24h)
}
```

Timeout serveur : L'outil `mcp__ide__execute_shell_command` n'est pas disponible dans cet environnement. Le code ci-dessus est une représentation du fonctionnement.

11. Design Responsive (Mobile & Desktop)

ViteComm suit une approche **mobile-first** avec le breakpoint `md:` (768px) comme seuil principal desktop.

11.1 Stratégie responsive

Élément	Mobile (<768px)	Desktop (≥768px)
Navigation	Menu hamburger + tiroir latéral (<code>MobileDrawer</code> , slide-out gauche)	Barre d'onglets horizontaux (<code>hidden md:flex</code>)
Header	Bouton menu + logo minimal	Onglets nav + avatar + nom + déconnexion
Grilles	<code>grid-cols-1</code> (empilé)	<code>sm:grid-cols-2</code> , <code>lg:grid-cols-3</code> , <code>md:grid-cols-4</code>
Hero (landing)	<code>text-3xl</code> , padding réduit	<code>text-5xl</code> / <code>text-6xl</code> , padding étendu
Boutons	Texte tronqué (<code>hidden sm:inline</code>), taille <code>sm</code>	Texte complet, taille normale
Footer	1 colonne	<code>grid-cols-1 md:grid-cols-3</code>
Admin stats	<code>grid-cols-2</code>	<code>md:grid-cols-4</code>

11.2 Détection du rôle pour le thème

Chaque layout (`ClientLayout` , `VendeurLayout` , `LivreurLayout`) ajoute une classe CSS sur l'élément `<html>` au montage :

```
// ClientLayout.jsx
useEffect(() => {
  document.documentElement.classList.add('role-client');
  return () => document.documentElement.classList.remove('role-client');
}, []);
```

Cela active les couleurs d'accent spécifiques au rôle dans `index.css` :

```
html.role-client { --accent: #1D9E75; } /* Vert */
html.role-vendeur { --accent: #BA7517; } /* Orange */
html.role-livreur { --accent: #D85A30; } /* Rouge */
```

12. Système de Thème (Clair / Sombre / Système)

12.1 Trois modes

Mode	Stockage	Comportement
light	Clé <code>vc_theme</code> dans localStorage	Force le thème clair
dark		Force le thème sombre (classe <code>.dark</code> sur <code><html></code>)
system		Suit le système via <code>prefers-color-scheme: dark</code> + listener en temps réel

12.2 Variables CSS

```
:root {
  --bg: #F7F8F3;          /* Fond principal */
  --surface: #FFFFFF;      /* Cartes, panels */
  --accent: #1D9E75;       /* Couleur d'accent (change selon le rôle) */
  --text-primary: #111827;
  --text-secondary: #4b5563;
  --text-muted: #9ca3af;
  --border: #e5e7eb;
}

.dark {
  --bg: #1A1B19;
  --surface: #252623;
  --accent: #2DC491;
  --text-primary: #f9fafb;
  --text-secondary: #d1d5db;
  --text-muted: #6b7280;
  --border: #374151;
}
```

12.3 Transitions

```
* {
  transition: background-color 0.25s ease, color 0.25s ease,
             border-color 0.25s ease, box-shadow 0.25s ease;
}
```

13. Progressive Web App (PWA)

13.1 Configuration

- **Plugin** : `vite-plugin-pwa` v1.3.0
- **Mode** : `generateSW` avec `registerType: 'autoUpdate'`
- **Manifest** : `display: standalone` , `orientation: portrait` , `theme_color: #1D9E75`
- **Icônes** : 192x192, 512x512, maskable 512x512 (logo V vert)

13.2 Mise en cache (Workbox)

Pattern	Stratégie	Cache
Google Fonts	<code>CacheFirst</code>	1 an
gstatic fonts	<code>CacheFirst</code>	1 an
<code>/api/*</code>	<code>NetworkFirst</code>	1h, timeout 10s
Assets statiques	Pré-cache (17 entrées, ~1.4 MB)	—

13.3 Composant InstallPrompt

Le composant `InstallPrompt.jsx` gère l'installation de la PWA :

```

// InstallPrompt.jsx – logique d'installation
const [deferredPrompt, setDeferredPrompt] = useState(null)
const [isStandalone] = useState(() =>
  window.matchMedia('(display-mode: standalone)').matches ||
  window.navigator.standalone === true
)

useEffect(() => {
  if (isStandalone) return // Déjà installée
  if (localStorage.getItem('pwa-install-dismissed')) return // Dismissed

  const handler = (e) => {
    e.preventDefault()
    setDeferredPrompt(e) // Stocke l'événement
    setTimeout(() => setShowModal(true), 2000) // Affiche après 2s
  }
  window.addEventListener('beforeinstallprompt', handler)
  return () => window.removeEventListener('beforeinstallprompt', handler)
}, [])

const handleInstall = async () => {
  deferredPrompt.prompt() // Déclenche le popup natif
  const { outcome } = await deferredPrompt.userChoice
  if (outcome === 'accepted') { /* installé */ }
}

```

- **Détection standalone** : `(display-mode: standalone)` ou `navigator.standalone` — si vrai, l'app est déjà installée, le composant ne rend rien
- **Bannière flottante** : Bouton violet en bas à droite avec animation bounce
- **Modal** : Slide-up avec backdrop blur — "Installer" ou "Pas maintenant"
- **Dismiss** : `localStorage.setItem('pwa-install-dismissed', 'true')` — ne plus afficher
- **Événement `appinstalled`** : Nettoie l'état après installation réussie

13.4 Avantages PWA pour ViteComm

- **Installation native** : L'utilisateur peut ajouter ViteComm à son écran d'accueil comme une app
- **Mode plein écran** : `display: standalone` — pas de barre d'adresse du navigateur
- **Orientation** : `portrait` — force l'orientation verticale
- **Fonctionnement hors-ligne** : Les assets statiques sont mis en cache (Workbox), les pages visitées restent accessibles
- **Performance** : Cache Google Fonts (1 an), cache API (1h), pré-cache des assets (~1.4 MB)
- **Mise à jour auto** : `registerType: 'autoUpdate'` — le service worker se met à jour silencieusement

14. Paiement Mobile Money (Fedapay)

14.1 Intégration

- **Provider** : Fedapay (MTN MoMo, Moov Pay, Celtis Cash)
- **Devise** : XOF (FCFA)
- **Commission** : 0.6% par transaction validée
- **Frais livraison** : 1500 F fixes
- **Frais retour** : 500 F par article rejeté

14.2 Flux de paiement

- 1 **Initiation** : Le client saisit son numéro de téléphone → `POST /api/client/payment/initiate` crée une `PaiementTransaction` et appelle l'API Fedapay → retourne une `checkout_url`.

Code – Création de la transaction (`paymentController.js`)

```
const montant = facture.montant_total_du;
const transaction_id = generateTransactionId(); // "TXN-XXXXXXXXXXXX"

// 1. Créer l'enregistrement en BDD
const transaction = await prisma.paiementTransaction.create({
  data: {
    id_user_client, id_commande, montant, mode_paiement,
    telephone, transaction_id, statut: 'pending',
  },
});

// 2. Appeler l'API Fedapay
const fedapayResult = await initiatePayment({
  id_commande, montant, devise: 'XOF', mode_paiement,
  telephone, transaction_id, client_prenom, client_nom,
});

// 3. Stocker l'ID Fedapay et retourner l'URL de checkout
await prisma.paiementTransaction.update({
  where: { id_paiement_transaction: transaction.id_paiement_transaction },
  data: { fedapay_transaction_id: fedapayResult.fedapay_id },
});

return res.status(201).json({
  checkout_url: fedapayResult.checkout_url, // → redirige le navigateur
  transaction_id,
});
```

Code – Appel API Fedapay (`fedapayService.js`)

```

const payload = {
  description: `Commande ViteComm #${id_commande}`,
  amount: Math.round(montant),
  currency: { iso: 'XOF' },
  reference: transaction_id,
  callback_url: `${APP_URL}/api/webhooks/fedapay`, // Webhook (POST serveur)
  return_url: `${FRONTEND_URL}/client/paiement?ref=...`, // Redirect (GET navigateur)
  customer: { firstname, lastname, phone_number: { number: phone, country: 'BJ' } },
  method: 'mtn_bj', // ou 'moov_bj', 'celtis_bj'
};

const response = await fetch(`${FEDAPAY_BASE_URL}/v1/transactions`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${SECRET_KEY}` },
  body: JSON.stringify(payload),
});

```

- 2 **Redirection** : Le client est redirigé vers le portail FedaPay, confirme sur son téléphone (code USSD).
- 3 **Webhook** : FedaPay envoie un POST signé (HMAC-SHA-256) à </api/webhooks/fedapay> . Le backend vérifie la signature (timing-safe, fenêtre 5 min) et met à jour le statut.

Code — Traitement webhook ([paymentController.js](#))

```

export const handleWebhook = async (req, res) => {
  // 1. Vérifier la signature HMAC
  const signatureHeader = req.headers['x-fedapay-signature'];
  const signature = verifyWebhookSignature(req.rawBody, signatureHeader);
  if (!signature) return res.status(400).json({ error: 'Signature invalide' });

  // 2. Extraire les données
  const event = req.body;
  const eventType = event.event; // "transaction.approved" | "transaction.declined"
  const transactionData = event.data?.transaction;

  // 3. Trouver la transaction en BDD
  const paiementTransaction = await prisma.paiementTransaction.findFirst({
    where: { OR: [
      { transaction_id: reference },
      { fedapay_transaction_id: String(transactionData.id) },
    ]},
  });

  // 4. Traiter selon le type d'événement
  if (eventType === 'transaction.approved') {
    await prisma.$transaction(async (tx) => {
      await tx.paiementTransaction.update({ data: { statut: 'completed' } });
      await tx.paiement.create({ data: { montant_percu, mode_reglement: 'MOBILE_MONEY' } });
      await tx.facture.update({ data: { statut_paiement: 'Payé' } });
      await tx.commande.update({ data: { mode_paiement_status: 'payé' } });
    });
  }
};

```

4 **Succès** : La facture est marquée "Payé", le gain du livreur est crédité. Le client voit la facture détaillée.

14.3 Fallback polling (quand le webhook échoue)

En dev local, le webhook ne peut pas atteindre votre IP privée. Le frontend utilise le **polling** comme fallback — il interroge directement l'API Fedapay depuis le backend :

```

// paymentController.js – getPaymentStatus
export const getPaymentStatus = async (req, res) => {
  const transaction = await prisma.paiementTransaction.findFirst({
    where: { transaction_id, id_user_client },
  });

  // Si encore en attente, vérifier directement auprès de Fedapay
  if (transaction.statut === 'pending' && transaction.fedapay_transaction_id) {
    const fedapayStatus = await verifyTransactionStatus(transaction.fedapay_transaction_id);
    // ↑ Appel GET /v1/transactions/:id vers Fedapay API

    if (fedapayStatus === 'approved') {
      // Simule le traitement du webhook
      await prisma.$transaction(async (tx) => {
        await tx.paiementTransaction.update({ data: { statut: 'completed' } });
        await tx.paiement.create({ data: { montant_percu, mode_reglement: 'MOBILE_MONEY' } });
        await tx.facture.update({ data: { statut_paiement: 'Paye' } });
        await tx.commande.update({ data: { mode_paiement_status: 'paye' } });
      });
    }
  }

  return res.json({ statut: transaction.statut, montant, mode_paiement });
};

// Appel API Fedapay pour vérifier le statut
export async function verifyTransactionStatus(fedapayId) {
  const response = await fetch(`${FEDAPAY_BASE_URL}/v1/transactions/${fedapayId}`, {
    headers: { Authorization: `Bearer ${FEDAPAY_SECRET_KEY}` },
  });
  const data = await response.json();
  return tx?.status; // "approved" | "declined" | "canceled" | "pending"
}

```

14.4 Sécurité webhook

```
// paymentController.js
const signature = req.headers['x-fedapay-signature'];
const expected = crypto.createHmac('sha256', WEBHOOK_SECRET)
    .update(JSON.stringify(req.body)).digest('hex');
const timestamp = req.headers['x-fedapay-timestamp'];
if (Math.abs(Date.now() - timestamp) > 300000) return res.status(400).send('Expired');
if (!crypto.timingSafeEqual(Buffer.from(signature), Buffer.from(expected)))
    return res.status(400).send('Invalid signature');
```

15. Upload de Fichiers et Preuves

15.1 Middleware Multer

Type	Dossier	Limite	Filtres
Avatar	uploads/avatars/	5 MB	Images uniquement
Image marché	uploads/markets/	5 MB	Images uniquement
Photo produit	uploads/products/	5 MB	Images uniquement
Preuve collecte	uploads/proofs/	10 MB	Images + vidéos

15.2 Processus d'upload

1. Le fichier est d'abord sauvegardé dans un dossier `temp/`
2. Après validation en base, `moveToPermanent()` déplace le fichier vers le dossier définitif
3. L'URL publique est retournée : `/uploads/<dossier>/<filename>`
4. Les fichiers sont servis statiquement : `app.use('/uploads', express.static(...))`

15.3 Preuves photo (RG07)

- **Livreur** : Photo obligatoire lors de la collecte chez le vendeur (max 5 photos)
- **Client** : Photo optionnelle lors de l'inspection (si litige, max 5 photos/vidéos)
- Chaque photo crée un enregistrement `MediaPreuve` lié à un `PreuveCollecte`

16. CORS et Gestion des Erreurs

16.1 CORS

Environnement	Comportement
Développement (<code>APP_ENV=local</code>)	<code>origin: true</code> — toutes les origines autorisées
Production	Whitelist depuis <code>CORS_ORIGINS</code> (virgule-séparé). Pas d'origine = autorisé (server-to-server).

16.2 Gestion des erreurs backend

- **Mode debug** (`APP_DEBUG=true`) : renvoie le message d'erreur brut

- **Mode production** (`APP_DEBUG=false`) : renvoie "Une erreur est survenue."
- **Transactions** : `prisma.$transaction` garantit l'atomicité des opérations multi-étapes
- **Error handler global** dans `index.js` : catch les erreurs non gérées, log le stack

16.3 Gestion des erreurs frontend

- **ErrorBoundary** : Composant class React qui catch les erreurs de render, affiche une carte d'erreur avec bouton "Réessayer"
- **API errors** : `api.js` lance des `Error` avec messages français ; le 401 redirige automatiquement vers `/connect`
- **Timeout** : 30 secondes via `AbortController`
- **Messages d'erreur par type** : erreur, suspendu, banni, réseau, Google — chacun avec icône/couleur différente

ViteComm — Guide technique généré automatiquement à partir de l'analyse du code source.

Stack : React 19 + Vite 8 + Tailwind CSS 4 + Node.js + Express 5 + Prisma 7 + MySQL/MariaDB

Juin 2026